

RAILCAB

Admin Panel — Complete Task Breakdown

Module-wise Task Allocation for 5-Member Team

Version 1.0 | March 2026 | CONFIDENTIAL

Prepared for: Novelski Technologies Pvt Ltd

5	10 Modules	4 Roles	16 Screens	40+ APIs
Team Members	Feature Modules	RBAC Roles	Admin Pages	Backend Endpoints

Table of Contents

1.	Team Overview & Assignment Matrix
2.	Module 1 – Super Admin Dashboard & Platform Config
3.	Module 2 – RBAC & Admin User Management
4.	Module 3 – Driver Onboarding & KYC Workflow
5.	Module 4 – Vendor Management & Fleet
6.	Module 5 – Ride Management
7.	Module 6 – Dynamic Fare Engine
8.	Module 7 – Finance & Accountant Module
9.	Module 8 – Support & Ticket Management
10.	Module 9 – Analytics & Reporting Dashboard
11.	Module 10 – Notifications & Communication
12.	Module 11 – Fleet & Vehicle Management
13.	Module 12 – Security, VAPT & Audit Logs
14.	GitHub Workflow — How to Merge All Modules into One Admin
15.	Appendix – API Quick Reference

1. Team Overview & Assignment Matrix

The RailCab Admin Panel is divided across 5 team members with clear module ownership. All Phase 1 features are in scope — no deferred work. Each member owns specific screens, backend routes, database models, and CRUD operations. The final admin panel is assembled via GitHub feature branches following the workflow in Section 14.

Member 1	Arjun Reddy	Backend Lead	Auth, RBAC, Super Admin, Security/VAPT, Audit
Member 2	Priya Sharma	Backend Developer	Driver Onboarding, Vendor Mgmt, Fleet Backend
Member 3	Kiran Patel	Backend Developer	Fare Engine, Finance/Accountant, Analytics, Notifications
Member 4	Sneha Iyer	Frontend Lead	Auth UI, Dashboard, Admin Users, Drivers, Vendors, Rides pages
Member 5	Vikram Nair	Frontend Developer	Finance, Support, Fare, Fleet, Analytics, Notifications, Audit, Settings

Assignment Matrix

Module	Backend Owner	Frontend Owner	Est. Days	Priority
Authentication & Session Mgmt	Arjun (M1)	Sneha (M4)	3d	Critical
RBAC & Admin User Management	Arjun (M1)	Sneha (M4)	3d	Critical
Super Admin Dashboard & Config	Arjun (M1)	Sneha (M4)	3d	High
Driver Onboarding & KYC	Priya (M2)	Sneha (M4)	5d	High
Vendor Management & Fleet Onboarding	Priya (M2)	Sneha (M4)	4d	High
Ride Management	Priya (M2)	Sneha (M4)	3d	High
Dynamic Fare Engine	Kiran (M3)	Vikram (M5)	4d	High
Finance & Accountant Module	Kiran (M3)	Vikram (M5)	5d	High
Support & Ticket Management	Kiran (M3)	Vikram (M5)	3d	Medium
Analytics & Reporting	Kiran (M3)	Vikram (M5)	3d	Medium
Notifications & Communication	Kiran (M3)	Vikram (M5)	2d	Medium
Fleet & Vehicle Management	Priya (M2)	Vikram (M5)	3d	Medium
Security, VAPT & Audit Logs	Arjun (M1)	Vikram (M5)	3d	Critical

Super Admin Dashboard, Auth & Platform Config

Team: Arjun (Backend)
+ Sneha (Frontend)

This module covers the authentication system, 2FA enforcement, JWT session management, platform-wide configuration, and the Super Admin dashboard with live KPIs. Only 1–3 Super Admin accounts may exist. All critical actions require 2FA and are logged immutably.

Task 1.1

Authentication System (Login, 2FA, Logout, Password Change)

Priority: Critical

Est: 3 days

Description

Build the complete auth flow: email/password login, TOTP 2FA verification, JWT access/refresh token issuance, secure httpOnly cookie storage, brute-force protection (5 attempts → 30 min lockout), and forced password change on first login. Implement token refresh and logout/session invalidation. The existing LoginPage.jsx and ChangePasswordPage.jsx stubs must be fully wired.

■ Screen Names & Navigation Path

- /login → LoginPage.jsx — Email + password form with 2FA TOTP step
- /change-password → ChangePasswordPage.jsx — Forced on first login; zxcvbn strength meter
- POST-login 2FA modal — 6-digit TOTP code entry with 5-attempt limit

Inputs	Outputs / Expected Results
• email (string), password (min 12 chars, uppercase, number, special char)	• access_token (JWT, 15 min expiry) stored in httpOnly cookie; user object returned with role & permissions
• totp_code (6-digit string from authenticator app)	• Session created; redirect to /dashboard on success; error on invalid code
• new_password + confirm_password on forced change	• Password updated; all existing sessions invalidated; redirect to login

■ API Endpoints

Method	Endpoint	Auth Required	Description
POST	/api/v1/auth/login	None	Validate credentials, check 2FA, return JWT
POST	/api/v1/auth/2fa/verify	Partial JWT	Verify TOTP code, issue full access token
POST	/api/v1/auth/refresh	Refresh token cookie	Issue new access token
POST	/api/v1/auth/logout	JWT	Blacklist token, clear cookie
POST	/api/v1/auth/change-password	JWT	Validate current password, set new, invalidate all sessions
GET	/api/v1/auth/me	JWT	Return current user profile with role and permissions
POST	/api/v1/auth/2fa/setup	JWT	Generate TOTP secret + QR code for setup

■ CRUD Operations Required

Operation	Resource / Model	Details
CREATE	AdminUser session / token	Issue JWT on login; store refresh token ref in Redis
READ	AdminUser	Fetch user by email for login; load permissions
UPDATE	AdminUser	Update password hash, last_login, must_change_password flag
DELETE	Redis session	Blacklist access token; remove refresh token on logout / password change

■ Acceptance Criteria / Definition of Done

- Login rejects invalid credentials with generic error (no user enumeration)
- 2FA blocks login if TOTP code incorrect; account locks after 5 attempts for 30 min
- JWT expires in 15 min; refresh token works for 7 days; httpOnly cookie set
- Changing password invalidates ALL existing sessions for that user
- First-login password change enforced before accessing any other page
- All login events (success + failure) written to audit_logs table

Notes: Backend: extend existing auth.py route. Frontend: wire AuthContext.jsx fully. Use speakeasy (Node) or pyotp (Python) for TOTP.

Task 1.2

Super Admin Dashboard — Live KPIs & Alerts

Priority: High

Est: 2 days

Description

Build the main dashboard that Super Admins and Managers see on login. Shows real-time platform metrics: active drivers, live rides, pending approvals, today's revenue, cancellation count, surge zone alerts, and low supply warnings. The existing DashboardPage.jsx stub must display real API data with auto-refresh every 60s.

■ Screen Names & Navigation Path

- /dashboard → DashboardPage.jsx — Full KPI grid with live tiles
- Alert banner component — Surge zone / low supply warnings at top
- Pending Approvals quick-action widget — Click to navigate to /drivers or /vendors

Inputs

- GET request to /admin/dashboard/summary (no body params)
- Optional: ?city=Hyderabad filter for city-scoped metrics

Outputs / Expected Results

- JSON response: live.active_drivers, live.live_rides, live.pending_requests, today.total_rides, today.revenue, today.cancellations, today.avg_fare, pending_approvals.drivers, pending_approvals.vendors, alerts[]
- Frontend renders color-coded KPI cards (green/amber/red), alert banners, and mini-charts for today's hourly ride volume

■ API Endpoints

Method	Endpoint	Auth Required	Description
GET	/api/v1/admin/dashboard/summary	JWT + Super Admin / Manager	Return live KPIs, today stats, alerts
GET	/api/v1/admin/dashboard/hourly-rides	JWT + Admin/Manager	Return ride count by hour for today

■ CRUD Operations Required

Operation	Resource / Model	Details
READ	Driver, Ride, Vendor, Config tables	Aggregate counts for KPIs via SQL COUNT / SUM
READ	Redis cache	Check driver socket presence cache for active driver count

■ Acceptance Criteria / Definition of Done

- Dashboard loads in under 2 seconds; stale data shown with warning if API fails
- KPI tiles auto-refresh every 60 seconds without full page reload
- Alert banners show surge zones and low-supply areas from real data
- Pending approvals count is clickable and navigates to the approval queue

Task 1.3

Platform Configuration Panel

Priority: High

Est: 2 days

Description

Super Admin-only page to GET and PATCH all global platform settings: commission percentages, ride search radius, OTP expiry, cancellation penalties, driver rating thresholds, safety deposit amount, and auto-suspend rules. Changes must be logged to audit_logs. Existing SettingsPage.jsx must be built out.

■ Screen Names & Navigation Path

- /settings → SettingsPage.jsx — Tabbed config form (General, Commission, Rides, Safety)
- Config Change Confirmation Modal — Shows before/after diff, requires 2FA re-confirmation

Inputs	Outputs / Expected Results
• platform_commission_pct (float 0–30)	• 200 response with full updated config object
• vendor_commission_pct (float 0–30)	• Audit log entry with before/after snapshot
• ride_search_radius_km (int 1–20)	• All active services notified of config update via Redis pub/sub
• otp_expiry_seconds (int 60–600)	•
• driver_cancel_penalty_inr (float)	•
• user_cancel_penalty_inr (float)	•
• min_driver_rating_threshold (float 1.0–5.0)	•
• max_drivers_per_ride_broadcast (int 1–20)	•
• safety_deposit_amount_inr (float)	•
• auto_suspend_rating_below (float)	•

■ API Endpoints

Method	Endpoint	Auth Required	Description
GET	/api/v1/admin/config	Super Admin JWT	Return full platform config
PATCH	/api/v1/admin/config	Super Admin JWT + 2FA	Update one or more config fields

■ CRUD Operations Required

Operation	Resource / Model	Details
READ	PlatformConfig	Fetch current config; return defaults if not set
UPDATE	PlatformConfig	PATCH single config row; store as JSON in DB
CREATE	AuditLog	Write before/after snapshot on every config change

■ Acceptance Criteria / Definition of Done

- Only Super Admin role can access GET and PATCH; 403 for all other roles
- Config changes confirmed by 2FA re-verification
- Audit log captures old value and new value for every changed field
- Frontend shows real-time validation (e.g., commission must be 0–30%)

RBAC — Roles & Admin User Management

Team: Arjun (Backend)
+ Sneha (Frontend)

Full role-based access control with granular permissions. Super Admin creates roles, assigns permissions, and provisions admin users. The existing AdminUsersPage.jsx has the UI stub; this task wires it fully to the backend and adds the Roles management view.

Task 2.1 Role Management — Create, Edit, Assign Permissions

Priority: Critical

Est: 2 days

Description Super Admin can CREATE new roles with arbitrary permission sets, READ existing roles, UPDATE permissions on a role, and DELETE unused roles. Built-in roles (super_admin, manager, accountant, support_agent, vendor, auditor) cannot be deleted. Each role has a permission bitmask or JSON array from the 11 defined permissions.

■ Screen Names & Navigation Path

- /admin-users → AdminUsersPage.jsx — Tabs: 'Users' and 'Roles'
- Roles tab: table of all roles with permission chips
- Create Role Modal — Name, description, checkbox grid of all 11 permissions
- Edit Role Modal — Same form pre-populated
- Delete Role Confirmation Dialog

Inputs	Outputs / Expected Results
• role_name (string), permissions (array of permission keys), description (string)	• role_id, role_name, permissions[], created_at, status
• created_by (auto from JWT), send_welcome_email (bool) on user creation	• 403 if non-super-admin attempts role creation

■ API Endpoints

Method	Endpoint	Auth Required	Description
POST	/api/v1/admin/roles	Super Admin	Create new role
GET	/api/v1/admin/roles	Admin JWT	List all roles with permissions
GET	/api/v1/admin/roles/:id	Admin JWT	Get single role detail
PATCH	/api/v1/admin/roles/:id	Super Admin	Update role name or permissions
DELETE	/api/v1/admin/roles/:id	Super Admin	Delete custom role (not built-in)

■ CRUD Operations Required

Operation	Resource / Model	Details
CREATE	Role	Insert role with permission JSON array
READ	Role	List all; filter by status; join to user count
UPDATE	Role	Patch permissions JSON; log before/after
DELETE	Role	Soft delete; reject if any admin user assigned to this role

■ Acceptance Criteria / Definition of Done

- 11 permission keys enforced: manage_roles, approve_drivers, approve_vendors, view_rides, manage_fares, process_payouts, view_financials, resolve_disputes, send_notifications, export_reports, system_config, audit_logs
- Built-in roles cannot be deleted
- Deleting a role with assigned users returns 409 Conflict

Task 2.2	Admin User Management — CRUD & Invite	Priority: Critical	Est: 2 days
----------	---------------------------------------	--------------------	-------------

Description Super Admin creates admin panel users by assigning a role and sending a welcome email with a temporary password. Managers can create Support Agent accounts. Users can be suspended (no login) or deleted. Password reset by admin sends a new temp password.

■ **Screen Names & Navigation Path**

- /admin-users → AdminUsersPage.jsx — Users tab: searchable table with status badges
- Invite User Modal — Name, email, phone, role dropdown, 'Send Welcome Email' toggle
- User Detail Drawer — Activity log, role, last login, suspend/delete actions
- Reset Password button — Sends new temp password via email

Inputs	Outputs / Expected Results
• name, email, phone (+91 format), role_id, created_by, send_welcome_email	• admin_user_id, name, email, role, temp_password_sent: true, must_change_password: true

■ **API Endpoints**

Method	Endpoint	Auth Required	Description
POST	/api/v1/admin/users	Super Admin	Create admin user & send invite
GET	/api/v1/admin/users	Super Admin	List all admin users, paginated
GET	/api/v1/admin/users/:id	Admin JWT	Get single user profile
PATCH	/api/v1/admin/users/:id	Super Admin	Update role, name, phone
DELETE	/api/v1/admin/users/:id	Super Admin	Soft-delete admin user
POST	/api/v1/admin/users/:id/suspend	Super Admin	Toggle account suspension
POST	/api/v1/admin/users/:id/reset-password	Super Admin	Send new temp password email

■ **CRUD Operations Required**

Operation	Resource / Model	Details
CREATE	AdminUser	Insert with hashed temp password, must_change_password=true
READ	AdminUser	List with role join; filter by role, status; paginate
UPDATE	AdminUser	Patch name/phone/role; log role changes to audit
DELETE	AdminUser	Soft delete (is_active=false); keep for audit trail

■ **Acceptance Criteria / Definition of Done**

- Welcome email contains temp password and must-change-on-login flag
- Suspended users get 403 on login attempts
- Super Admin cannot delete their own account
- Role change is logged with before/after in audit_logs

Driver Onboarding, KYC & Approval Workflow

Team: Priya (Backend)
+ Sneha (Frontend)

Complete 8-step driver onboarding: OTP registration → personal info → 7-document KYC upload → AI pre-check (OCR) → human KYC review → safety deposit → BGV → activation. The existing DriversPage.jsx stub must show the full approval queue, driver detail drawer, approve/reject workflows with reason codes, and status state machine.

Task 3.1 Driver Self-Registration & OTP (Driver App API)

Priority: High

Est: 2 days

Description Driver registers via mobile app: send OTP to phone, verify OTP, submit personal info and vehicle details. The backend creates a driver record at status=registered. The admin panel exposes a read-only view of all registered drivers.

■ Screen Names & Navigation Path

- (Mobile App) Onboarding Step 1: Phone OTP Entry
- (Mobile App) Onboarding Step 2: Personal Info Form (name, DOB, city, vehicle details)
- /drivers → DriversPage.jsx — Filter by status: shows all statuses including 'registered'

Inputs	Outputs / Expected Results
• phone (+91 format), otp (6-digit), name, dob (YYYY-MM-DD), city, vehicle_type (enum), vehicle_number, vehicle_model, referral_code (optional)	• driver_id, status: 'registered', onboarding_step: 2, next_action: 'upload_documents'

■ API Endpoints

Method	Endpoint	Auth Required	Description
POST	/api/v1/driver/send-otp	None	Generate & SMS 6-digit OTP; expire 5 min
POST	/api/v1/driver/verify-otp	None	Verify OTP; return temp session token
POST	/api/v1/driver/register	Temp JWT	Submit personal + vehicle info; create Driver record
GET	/api/v1/admin/drivers	Manager+	List all drivers with filters: status, city, rating

■ CRUD Operations Required

Operation	Resource / Model	Details
CREATE	Driver	Insert new driver row with status=registered
CREATE	OTPRecord	Store OTP hash + expiry in Redis; single-use
READ	Driver	Admin list with pagination, filter by status/city/rating
UPDATE	Driver	Update onboarding_step after each completed step

■ Acceptance Criteria / Definition of Done

- OTP is 6 digits, expires in 5 minutes, single-use
- Duplicate phone number returns 409 Conflict
- Admin can list drivers filtered by status, city, vehicle type
- Vehicle number must match regex: 2 letters + 2 digits + 2 letters + 4 digits (Indian format)

Task 3.2 Document Upload & AI Pre-Check

Priority: High

Est: 2 days

Description Driver uploads 7 KYC documents (Aadhaar front/back, PAN, Driving Licence, RC, Insurance, Fitness Certificate, Profile Photo) as S3 presigned URLs. After upload, the backend runs an OCR-based auto-check: name matching across documents, expiry validation on DL/Insurance/RC, face clarity check on profile photo. Results are stored per-document in the driver.documents JSON field.

■ **Screen Names & Navigation Path**

- (Mobile App) Document Upload Screen — 8-slot grid showing upload status per doc
- /drivers/:id → Driver Detail Drawer — 'Documents' tab showing all docs with AI check results
- Document Viewer Modal — Full-screen presigned URL display with zoom

Inputs	Outputs / Expected Results
• documents[] array: [{type, url}] where type is one of: aadhaar_front, aadhaar_back, pan, driving_licence, rc, vehicle_insurance, fitness_certificate, profile_pic	• status: 'docs_uploaded', auto_check_results per doc: {name_extracted, match, expiry, valid}, human_review_eta_hours: 24

■ **API Endpoints**

Method	Endpoint	Auth Required	Description
POST	/api/v1/driver/:id/documents	Driver JWT	Submit S3 URLs for all 7 docs; trigger AI pre-check
GET	/api/v1/driver/:id/onboarding-status	Driver JWT	Return current step, status, doc check results
POST	/api/v1/admin/drivers/:id/request-reupload	Manager+	Flag specific docs for re-upload with reason

■ **CRUD Operations Required**

Operation	Resource / Model	Details
UPDATE	Driver.documents (JSON)	Store each doc URL + AI check result in documents JSON field
UPDATE	Driver.status	Transition to docs_uploaded after all 7 docs submitted
READ	Driver	Admin fetch single driver with all document URLs

■ **Acceptance Criteria / Definition of Done**

- All 7 document types required before status advances to docs_uploaded
- AI check compares name on Aadhaar vs name on Driving Licence (fuzzy match, threshold 80%)
- DL expiry date must be in the future; expired DL auto-rejects with reason code DL_EXPIRED
- Profile photo face detection: reject if no face detected or image is blurry
- Presigned URLs expire in 1 hour; admin viewers use 10-min expiry URLs

Task 3.3	Human KYC Review, Approval & Rejection Workflow	Priority: High	Est: 2 days
----------	--	----------------	-------------

Description Manager reviews each driver's KYC submission side-by-side: document images on left, AI check results on right. Can approve all docs, reject specific docs with reason codes, or request re-upload. On full approval, driver advances to deposit_pending. On rejection, driver is notified via SMS with specific docs to re-upload.

■ **Screen Names & Navigation Path**

- /drivers → DriversPage.jsx — 'Pending Approval' queue tab (filterable)
- Driver KYC Review Drawer — Side-by-side doc viewer + AI results + approve/reject controls
- Reject Modal — Reason code dropdown (DOCUMENT_MISMATCH, EXPIRED_DL, BLURRY_IMAGE, etc.) + free text
- Approve Confirmation Modal — BGV cleared toggle + notes field
- Driver Detail Page /drivers/:id — Full profile with status timeline

Inputs	Outputs / Expected Results
• action: 'approve' 'reject', reviewed_by (auto from JWT), notes (string)	• Approve: driver.status → docs_verified; notification_sent: true to driver
• For reject: reason_code (enum), reason_detail (string), documents_to_reupload (string[])	• Reject: driver.status → docs_rejected; can_reapply_after (7 days); SMS sent
• bgv_cleared: boolean (on approve)	•

■ API Endpoints

Method	Endpoint	Auth Required	Description
GET	/api/v1/admin/drivers/pending-approval	Manager+	List drivers awaiting KYC review
POST	/api/v1/admin/drivers/:id/approve	Manager+	Approve driver KYC
POST	/api/v1/admin/drivers/:id/reject	Manager+	Reject with reason codes
PATCH	/api/v1/admin/drivers/:id/suspend	Manager+	Suspend active driver
PATCH	/api/v1/admin/drivers/:id/activate	Manager+	Reactivate suspended driver

■ CRUD Operations Required

Operation	Resource / Model	Details
READ	Driver	Fetch pending queue (status=under_review); paginate; filter
UPDATE	Driver	Update status, reviewed_by, reviewed_at, rejection_reason, can_reapply_after
CREATE	AuditLog	Log every approve/reject action with before/after status
CREATE	Notification	SMS/push to driver on approval or rejection with details

■ Acceptance Criteria / Definition of Done

- Approved driver receives SMS: 'Your KYC has been approved. Please pay the safety deposit.'
- Rejected driver receives SMS listing exact documents to re-upload
- KYC review timestamps and reviewer ID are stored immutably
- Bulk approve not allowed — each driver reviewed individually
- Manager cannot approve their own account (driver_id != reviewer_id check)

Task 3.4	Safety Deposit & BGV → Account Activation	Priority: High	Est: 1 day
Description	After KYC approval, driver pays safety deposit via Razorpay. Backend records the payment, advances status to bgv_in_progress. Manager marks BGV cleared after police verification. System then activates the driver account (status=active) and unlocks the driver app.		
■ Screen Names & Navigation Path			
• (Mobile App) Safety Deposit Payment Screen — Razorpay checkout • /drivers/:id → Driver Drawer — 'Safety Deposit' badge + 'BGV Status' badge • BGV Approval button (Manager only) — with police verification reference field			
Inputs		Outputs / Expected Results	
• razorpay_payment_id (from Razorpay webhook), amount, driver_id		• Status: deposit_pending → bgv_in_progress after payment	
• bgv_cleared: true, bgv_reference: string (police verification ID)		• Status: bgv_in_progress → active after BGV clearance	
■ API Endpoints			

Method	Endpoint	Auth Required	Description
POST	/api/v1/driver/:id/safety-deposit	Driver JWT	Record Razorpay payment; advance status
POST	/api/v1/admin/drivers/:id/clear-bgv	Manager+	Mark BGV cleared; activate driver account
POST	/api/v1/payments/razorpay-webhook	None (HMAC)	Handle Razorpay payment success/failure

■ CRUD Operations Required

Operation	Resource / Model	Details
UPDATE	Driver	Set safety_deposit_status=completed, safety_deposit_razorpay_id
UPDATE	Driver	Set bgv_cleared=true, status=active, last_active_at
CREATE	DepositRecord	Create deposit transaction record with amount, razorpay_id, timestamp
CREATE	AuditLog	Log activation event

■ Acceptance Criteria / Definition of Done

- Razorpay webhook validated via HMAC signature before processing
- Double payment rejected if deposit already completed
- Driver app access denied until status=active
- Safety deposit amount configurable via /admin/config (default ■2000)

Vendor Management — Fleet Onboarding & Portal

Team: Priya (Backend)
+ Sneha (Frontend)

Vendors are fleet operators with 10–40 vehicles. They self-register, upload business documents, get Admin/Manager approval, then manage their fleet via the Vendor Portal. The existing VendorsPage.jsx stub must be built out fully.

Task 4.1 Vendor Registration, Document Upload & Admin Approval

Priority: High

Est: 2 days

Description Vendor self-registers with business name, GST, PAN, bank details, expected fleet size. Uploads business documents (GST cert, PAN, trade licence, cancelled cheque, fleet agreement). Admin/Manager reviews and approves or rejects. On approval, vendor gets access to their fleet dashboard.

■ Screen Names & Navigation Path

- (Vendor Portal) /vendor/register — Multi-step registration form
- (Vendor Portal) /vendor/documents — Document upload with progress tracker
- /vendors → VendorsPage.jsx — List of vendors with status, fleet size, earnings MTD
- Vendor Detail Drawer — Business info, documents tab, approve/reject controls
- Vendor Approval Modal — Approve with notes OR reject with reason

Inputs	Outputs / Expected Results
• business_name, owner_name, email, phone, gst_number (15-char regex), pan_number, city, expected_fleet_size (10–40), vehicle_types[], bank_account {account_number, ifsc, account_name}	• vendor_id, status: 'pending_review', application_number (VND-YYYY-NNNN), estimated_review_days: 2, documents_upload_url
• Document uploads: GST cert URL, PAN URL, trade licence URL, bank cheque URL, fleet agreement URL	• Email confirmation to vendor; admin notification of new vendor application

■ API Endpoints

Method	Endpoint	Auth Required	Description
POST	/api/v1/vendor/register	None	Vendor self-registration
POST	/api/v1/vendor/:id/documents	Vendor JWT	Upload business documents
GET	/api/v1/admin/vendors	Manager+	List all vendors paginated
GET	/api/v1/admin/vendors/:id	Manager+	Get vendor detail
POST	/api/v1/admin/vendors/:id/approve	Manager+	Approve vendor
POST	/api/v1/admin/vendors/:id/reject	Manager+	Reject vendor with reason
PATCH	/api/v1/admin/vendors/:id/suspend	Manager+	Suspend vendor account

■ CRUD Operations Required

Operation	Resource / Model	Details
CREATE	Vendor	Insert vendor row with status=pending_review
UPDATE	Vendor	Store document URLs; update on approval/rejection
READ	Vendor	Admin list with fleet size, earnings, status; filter by city/status
UPDATE	Vendor.status	Transition pending_review → active rejected

■ Acceptance Criteria / Definition of Done

- GST number validated against 15-char format: 2 digits + 10 alphanumeric + 1 char + 1 char + 1 digit
- IFSC code validated against 11-char format
- Vendor portal access blocked until status=active
- Maximum 40 vehicles per vendor enforced at fleet addition

Task 4.2 Fleet Management — Add Vehicles, Assign Drivers Priority: High Est: 2 days

Description Approved vendor can add 10–40 vehicles to their fleet (vehicle number, model, type, RC, insurance, fitness cert with expiry dates). Each vehicle requires admin verification. Vendor assigns/unlinks drivers to vehicles. Admin sees all vehicles in the Vehicle Registry (/fleet).

■ Screen Names & Navigation Path

- (Vendor Portal) /vendor/fleet → Fleet Management Page — Vehicle list, add/remove, live status
- (Vendor Portal) /vendor/fleet/add → Add Vehicle Form
- (Vendor Portal) /vendor/fleet/:id/assign-driver → Driver assignment select
- /fleet → FleetPage.jsx — All platform vehicles; filter by vendor/type/city/status

Inputs	Outputs / Expected Results
• vehicle_number (regex), vehicle_type, vehicle_model, year_of_manufacture, seating_capacity, fuel_type, rc_document_url, insurance_url, insurance_expiry (date), fitness_expiry (date)	• vehicle_id, status: 'pending_verification', vendor_id, safety_deposit_required: 2000.00
• driver_id to assign to vehicle	• Driver-vehicle link created; previous assignment auto-unlinked

■ API Endpoints

Method	Endpoint	Auth Required	Description
POST	/api/v1/vendor/:id/vehicles	Vendor JWT	Add vehicle to fleet
GET	/api/v1/vendor/:id/vehicles	Vendor JWT	List vendor's vehicles
PATCH	/api/v1/vendor/:id/vehicles/:vid	Vendor JWT	Update vehicle details
DELETE	/api/v1/vendor/:id/vehicles/:vid	Vendor JWT	Remove vehicle from fleet
POST	/api/v1/vendor/:id/vehicles/:vid/assign	Vendor JWT	Assign driver to vehicle
GET	/api/v1/admin/vehicles	Manager+	List all platform vehicles
POST	/api/v1/admin/vehicles/:id/verify	Manager+	Admin verify vehicle documents

■ CRUD Operations Required

Operation	Resource / Model	Details
CREATE	Vehicle	Insert vehicle with status=pending_verification
READ	Vehicle	Vendor: own fleet only; Admin: all fleet with filters
UPDATE	Vehicle	Update docs, expiry dates, status, current_driver
DELETE	Vehicle	Soft-delete (status=inactive); unlink any assigned driver
CREATE	DriverVehicleLink	Create assignment record; set end_time on previous link

■ Acceptance Criteria / Definition of Done

- Vehicle count per vendor capped at 40; returns 400 if exceeded
- Vehicle number uniqueness enforced platform-wide
- Insurance expiry < 30 days triggers maintenance alert
- Driver can only be assigned to one vehicle at a time

Admin view of all rides across the platform: live, completed, cancelled. Managers can force-complete stuck rides, reassign drivers, or cancel with penalty. The existing RidesPage.jsx stub must be wired to real data with full filter support.

Task 5.1 Ride List, Detail & Admin Actions

Priority: High

Est: 2 days

Description Admin/Manager sees all rides with filters: status, city, date range, vehicle type, driver, user. Can click into any ride for full detail: pickup/drop, fare breakdown, map route, driver info, user info, timeline of events. Admin actions: force-complete, force-cancel, reassign driver.

Screen Names & Navigation Path

- /rides → RidesPage.jsx — Filterable data table (status, city, date, vehicle type)
- Ride Detail Drawer — Full timeline, fare breakdown, map route (polyline), user/driver cards
- Force-Complete Modal — Reason field + override fare option
- Reassign Driver Modal — Search active nearby drivers, select new driver
- Cancel Ride Modal — Reason + penalty applicability toggle

Inputs	Outputs / Expected Results
• Filters: ?status=&city=&from=&to=&vehicle_type=&driver_id=&user_id=&page=&limit=	• rides[]: ride_id, user (id, name, phone), driver (id, name, vehicle), pickup, drop, distance_km, fare, status, vehicle_type, created_at — paginated
• Force-complete: reason, override_fare (optional)	• Ride detail: full fare breakdown, polyline, timestamps for each status change
• Reassign: new_driver_id, reason	•
• Cancel: reason, apply_penalty (bool)	•

API Endpoints

Method	Endpoint	Auth Required	Description
GET	/api/v1/admin/rides	Admin/Manager/Accountant/Support	List rides with filters
GET	/api/v1/admin/rides/:id	Admin/Manager/Support	Full ride detail
POST	/api/v1/admin/rides/:id/force-complete	Manager+	Force end a stuck ride
POST	/api/v1/admin/rides/:id/reassign	Manager+	Reassign to new driver
POST	/api/v1/admin/rides/:id/cancel	Manager+	Cancel ride with/without penalty

CRUD Operations Required

Operation	Resource / Model	Details
READ	Ride / TripHistory	Paginated query with multi-column filter; join User + Driver + Vehicle
UPDATE	Ride	Update status, end_time, override_fare on force-complete/cancel
CREATE	AuditLog	Log every admin action on a ride
CREATE	Notification	Notify driver and user of admin-forced status change

Acceptance Criteria / Definition of Done

- Rides table supports real-time filter updates without full page reload
- Force-complete calculates correct partial fare if distance is known

- Reassign only works if new driver is online and not on another trip
 - All admin actions on rides create an audit log entry
-

Dynamic Fare Engine — Configuration & Computation

Team: Kiran (Backend)
+ Vikram (Frontend)

The fare engine computes: Base Fare + Distance Fare + Time Fare, multiplied by Surge, Night Charge, Weather Factor, Traffic Factor. All parameters are admin-configurable. Existing FareEnginePage.jsx must allow Managers to view and edit all fare variables.

Task 6.1 Fare Configuration — Vehicle Rates, Surge, Night, Geo-Zones

Priority: High

Est: 2 days

Description Manager configures all fare variables per vehicle type. The backend stores config in a FareConfig table and exposes GET/PATCH endpoints. Frontend shows a tabbed form: 'Base Rates', 'Surge Settings', 'Night/Holiday Charges', 'Geo-Zone Overrides'. Changes take effect at a configurable future timestamp (effective_from).

■ Screen Names & Navigation Path

- /fare-engine → FareEnginePage.jsx — Tabbed fare config form
- 'Base Rates' tab — Per vehicle type: base fare, per_km_rate, per_min_rate, minimum fare
- 'Surge Settings' tab — max_surge cap, activation threshold, k_factor, inflection point
- 'Night/Holiday' tab — Start/end times, multipliers, enable/disable toggles
- 'Geo-Zones' tab — Map with polygon drawing tool; per-zone override fares (airport, station)
- Change Preview Modal — Shows current vs proposed fare for sample ride

Inputs	Outputs / Expected Results
• vehicle_rates: {Cab Economy: {base_fare, per_km, per_min}, Bike: {...}, ...}	• 200 with updated fare config; effective_from stored; current config returned until effective_from
• surge_config: {max_surge (1.0–5.0), activation_threshold, k_factor, inflection}	• Audit log entry with old vs new config snapshot
• night_charge: {enabled, start (HH:MM), end (HH:MM), multiplier (1.0–2.0)}	•
• geo_zone_overrides: [{zone_name, polygon_coords[], base_fare_override, per_km_override}]	•
• effective_from (ISO datetime)	•

■ API Endpoints

Method	Endpoint	Auth Required	Description
GET	/api/v1/admin/fare-config	Manager+	Return full current fare config
PATCH	/api/v1/admin/fare-config	Manager+	Update fare variables with effective_from
POST	/api/v1/booking/calculate-fare	None	Calculate fare for given pickup/drop/vehicle
GET	/api/v1/admin/fare-config/history	Manager+	Audit trail of past fare config changes

■ CRUD Operations Required

Operation	Resource / Model	Details
READ	FareConfig	Return active config (effective_from <= now)
CREATE	FareConfig	Insert new config version with effective_from
READ	GeoZone	Fetch all zone polygons for map display
CREATE	GeoZone	Insert new zone polygon with override fares

Operation	Resource / Model	Details
UPDATE	GeoZone	Update zone polygon or fare overrides
DELETE	GeoZone	Remove geo-zone override (revert to base rates)

■ Acceptance Criteria / Definition of Done

- Fare config versioned — old configs retained for historical fare calculations
- effective_from must be in the future (at least 5 minutes ahead)
- Calculate-fare API returns breakdown: base, distance, time, surge, night, tolls, platform_fee
- Max surge hard cap (from config) cannot be exceeded even by direct DB edit
- Geo-zone polygon overlap detection: warn manager if zones overlap

Task 6.2	Surge Algorithm — Supply/Demand Computation Service	Priority: High	Est: 2 days
----------	--	----------------	-------------

Description	Background microservice that runs every 5 minutes: counts active driver sockets per 2km×2km geo-cell, counts active ride requests, computes D/S ratio, applies sigmoid transformation to get surge multiplier (1.0–max_surge), emits surge zone events via Redis pub/sub to Socket.IO. Admin sees live surge map and can manually override surge for a zone.
--------------------	--

■ Screen Names & Navigation Path

- /fare-engine → Surge Map tab — Color-coded heat map of surge multipliers by zone
- Manual Surge Override Panel — Select zone, set multiplier, set expiry duration
- Surge History Chart — Timeline of surge events with demand/supply ratio

Inputs	Outputs / Expected Results
• Background: driver socket counts from Redis, ride request counts from DB	• surge_multiplier per geo-cell stored in Redis with 6-minute TTL
• Manual override: zone_id, multiplier (1.0–max_surge), expires_at (ISO datetime)	• Socket.IO event 'surge_update' emitted to all driver apps with new multiplier

■ API Endpoints

Method	Endpoint	Auth Required	Description
GET	/api/v1/admin/surge/live-map	Manager+	Return current surge multipliers by geo-cell
POST	/api/v1/admin/surge/override	Manager+	Set manual surge override for a zone
DELETE	/api/v1/admin/surge/override/:id	Manager+	Remove manual override (revert to algorithm)

■ CRUD Operations Required

Operation	Resource / Model	Details
READ	Redis surge cache	Get/set surge multiplier per geo-cell key
CREATE	SurgeOverride	Store manual override with expiry
DELETE	SurgeOverride	Remove manual override; algorithm resumes
CREATE	SurgeHistory	Log each surge computation event for analytics

■ Acceptance Criteria / Definition of Done

- Surge computation runs as a background Celery/APScheduler task every 5 minutes
- Manual override respected until expires_at; then algorithm takes over
- Max surge cap from FareConfig always enforced; cannot exceed via override
- Live map renders in under 1 second using cached Redis data

Complete financial management: driver earnings ledgers, vendor settlements, platform commission breakdowns, GST reports, payout initiation via Razorpay, invoice generation, safety deposit tracking, and dispute ledger. Accountants have read-only access to rides; no operational write access. Existing FinancePage.jsx must be built into a full financial suite.

Task 7.1

Driver Earnings Ledger & Payout Management

Priority: High

Est: 3 days

Description

Accountant views per-driver earnings: gross earnings, platform commission (15%), GST on commission (18%), net payable, already paid, pending payout — for any selected date range. Initiates bulk payout via Razorpay Payout API; batch records pending/processing/completed status. Dispute flagging adds a note to a specific ride earnings.

■ Screen Names & Navigation Path

- /finance → FinancePage.jsx — Tabs: Earnings, Settlements, Payouts, GST, Safety Deposits, Disputes
- 'Earnings' tab — Driver search; date-range picker; per-driver summary card + ride-level breakdown table
- 'Payouts' tab — Initiate payout: select driver(s), period, mode (NEFT/Razorpay); bulk select
- Payout Batch Detail Modal — Shows batch_id, total amount, count, status, ETA
- Dispute Flag Modal — Select ride_id; dispute reason; disputed amount

Inputs	Outputs / Expected Results
• driver_id (or vendor_id), from_date, to_date for earnings query	• Driver earnings: total_rides, gross_earnings, platform_commission, gst_on_commission, net_payable, already_paid, pending_payout, rides[]
• payout_type: 'driver' 'vendor', beneficiary_ids[], period_from, period_to, mode, initiated_by	• Payout initiation: batch_id, total_amount, count, status: 'processing', estimated_completion

■ API Endpoints

Method	Endpoint	Auth Required	Description
GET	/api/v1/admin/earnings/driver/:id	Accountant+	Driver earnings for period
GET	/api/v1/admin/earnings/vendor/:id	Accountant+	Vendor earnings for period
POST	/api/v1/admin/payouts/initiate	Accountant+	Initiate Razorpay bulk payout
GET	/api/v1/admin/payouts	Accountant+	List all payout batches
GET	/api/v1/admin/payouts/:batch_id	Accountant+	Payout batch status
POST	/api/v1/admin/earnings/flag-dispute	Accountant+	Flag ride earning as disputed

■ CRUD Operations Required

Operation	Resource / Model	Details
READ	Ride/TripHistory	Aggregate earnings per driver/vendor for period
CREATE	PayoutBatch	Insert batch record with status=processing
UPDATE	PayoutBatch	Update status (processing→completed/failed) via Razorpay webhook
CREATE	EarningsDispute	Insert dispute record for specific ride
READ	PayoutBatch	List batches with filter; paginate

■ Acceptance Criteria / Definition of Done

- Commission calculation: $\text{gross} \times \text{platform_commission_pct} / 100$
- GST calculation: $\text{commission} \times 18 / 100$
- Net payable = gross - commission - GST on commission
- Razorpay payout uses Fund Account API; IMPS for amounts < ₹2L, NEFT for larger
- Already-disputed rides cannot be included in a payout until dispute resolved

Task 7.2

GST Report, Invoice Generator & Audit Export

Priority: High

Est: 2 days

Description

Generate monthly GST-R1 ready data: total taxable turnover, output GST (18%), input tax credit, net GST payable, breakdown by service. Auto-generate weekly PDF invoices for drivers and vendors. Full transaction export as CSV/Excel for CA/auditor.

■ Screen Names & Navigation Path

- 'GST' tab — Month picker; GST summary card; download GST report (CSV)
- 'Invoices' tab — Search driver/vendor; select week; 'Generate Invoice' button → PDF download
- 'Audit Export' tab — Date range; entity type filter; 'Export CSV' and 'Export Excel' buttons

Inputs	Outputs / Expected Results
• month (YYYY-MM) for GST report	• GST report: JSON + CSV download
• driver_id or vendor_id + week_start_date for invoice generation	• Invoice: PDF with header (RailCab GSTIN, logo), earnings table, GST breakdown, payment history
• date_range + entity_type for audit export	• Audit export: CSV/Excel with all transaction rows

■ API Endpoints

Method	Endpoint	Auth Required	Description
GET	/api/v1/admin/gst-report	Accountant+	Monthly GST summary + breakdown
POST	/api/v1/admin/invoices/generate	Accountant+	Generate driver/vendor invoice PDF
GET	/api/v1/admin/invoices/:id/pdf	Accountant+	Download generated invoice as PDF
GET	/api/v1/admin/audit-export	Accountant+	Export full transaction CSV/Excel

■ CRUD Operations Required

Operation	Resource / Model	Details
READ	Ride/TripHistory	Aggregate for GST report: group by service type, sum fares
CREATE	Invoice	Generate invoice record; store PDF to S3
READ	Invoice	Fetch PDF from S3; return presigned URL
READ	All transaction tables	Full export JOIN across rides, earnings, payouts

■ Acceptance Criteria / Definition of Done

- GST report GSTIN matches platform config (36AABCT1332L1ZV or configured value)
- Invoice PDF renders: logo, invoice number, period, driver/vendor details, ride table, totals, GST line
- Export CSV contains: ride_id, date, driver, user, pickup, drop, fare, commission, net, gst, status
- Invoice generation completes in under 10 seconds even for 500+ rides

Support agents handle user/driver complaints, ride disputes, and refunds. Ticketing system with SLA tracking, priority queues, escalation to manager, and Razorpay refund initiation. Existing SupportPage.jsx stub to be fully built.

Task 8.1 Ticket Queue, Detail & Resolution Workflow

Priority: Medium

Est: 2 days

Description Support agents see a prioritized ticket queue (High/Medium/Low) with SLA breach warnings (red if deadline passed). Ticket detail shows conversation thread, linked ride details, attachments. Can resolve, escalate to manager, or create refund request. All agents see only assigned tickets by default; manager sees all.

Screen Names & Navigation Path

- /support → SupportPage.jsx — Ticket queue table with SLA timer, priority badges
- Ticket Detail Panel — Conversation thread; linked ride card; action buttons
- Resolve Ticket Modal — Resolution text; action_taken (refund/warning/none); refund amount
- Escalate Modal — Select escalation target (manager); add escalation note
- Create Ticket Modal — Raised by (user/driver); ride_id; issue_type; priority; description; attachments
- Refund Management tab — Pending refunds; approve/reject with amount

Inputs	Outputs / Expected Results
• raised_by_type, raised_by_id, ride_id, issue_type (overcharged/cancelled/safety/other), description, priority (high/medium/low), attachments[]	• ticket_id (TKT-YYYY-NNNNN), status: 'open', assigned_to, sla_deadline, created_at
• Resolve: resolution text, action_taken, refund_amount, internal_notes	• Resolve: status: 'resolved', refund_initiated: bool, refund_id, resolved_at

API Endpoints

Method	Endpoint	Auth Required	Description
POST	/api/v1/admin/support/tickets	Support+	Create new ticket
GET	/api/v1/admin/support/tickets	Support+	List tickets with filters
GET	/api/v1/admin/support/tickets/:id	Support+	Ticket detail with thread
POST	/api/v1/admin/support/tickets/:id/resolve	Support+	Resolve ticket + optional refund
POST	/api/v1/admin/support/tickets/:id/escalate	Support+	Escalate to manager
GET	/api/v1/admin/support/sla-report	Manager+	SLA breach analytics

CRUD Operations Required

Operation	Resource / Model	Details
CREATE	SupportTicket	Insert ticket; auto-assign to least-loaded support agent
READ	SupportTicket	Paginated list with priority sort; filter by type/status/assignee
UPDATE	SupportTicket	Resolve (update status, resolution, resolved_by, resolved_at)
CREATE	RefundRequest	Create refund record; trigger Razorpay reversal on approval
CREATE	TicketMessage	Thread message per conversation entry

■ Acceptance Criteria / Definition of Done

- SLA deadline: High priority = 4 hours; Medium = 12 hours; Low = 48 hours
- SLA breach turns ticket row red; sends email to manager
- Refund amount cannot exceed original fare
- Razorpay refund initiated immediately on resolve with action_taken='refund'
- All ticket actions logged with timestamp and agent_id

Comprehensive analytics for all roles: revenue, ride metrics, driver performance, vendor utilisation, zone demand, customer retention, support SLA, and compliance. Existing AnalyticsPage.jsx stub must show interactive charts with date range pickers.

Task 9.1 Revenue, Ride & Driver Performance Reports

Priority: Medium

Est: 3 days

Description Revenue report: gross/net/commission/GST for any period with day/week/month grouping and period comparison. Ride analytics: total rides, cancellation rate by type, avg fare, peak hours heatmap. Driver performance: rides/driver, rating distribution, online hours, acceptance rate — sortable table.

■ Screen Names & Navigation Path

- /analytics → AnalyticsPage.jsx — Tab bar: Revenue, Rides, Drivers, Vendors, Zones, Customers
- 'Revenue' tab — Line chart (period comparison), P&L; summary cards, CSV export
- 'Rides' tab — Bar chart (daily rides), cancellation donut, peak hours heatmap (24×7 grid)
- 'Drivers' tab — Sortable driver performance table; filter by city/vehicle type

Inputs	Outputs / Expected Results
• from_date, to_date, group_by (day/week/month), city (optional)	• Revenue: gross_revenue, net_revenue, platform_commission, gst_collected, prev_period_comparison_%
• Compare period: previous_period_from, previous_period_to	• Rides: total_rides, completed, cancelled_user%, cancelled_driver%, avg_fare, peak_hours[]

■ API Endpoints

Method	Endpoint	Auth Required	Description
GET	/api/v1/admin/analytics/revenue	Admin/Manager/Accountant	Revenue aggregates
GET	/api/v1/admin/analytics/rides	Admin/Manager	Ride metrics
GET	/api/v1/admin/analytics/drivers	Admin/Manager	Driver performance table
GET	/api/v1/admin/analytics/vendors	Admin/Accountant/Vendor	Vendor fleet utilisation
GET	/api/v1/admin/analytics/zones	Admin/Manager	Demand/supply by zone
GET	/api/v1/admin/analytics/customers	Admin/Manager	DAU, MAU, retention, LTV
GET	/api/v1/admin/analytics/export	Admin/Manager/Accountant	CSV/Excel export

■ CRUD Operations Required

Operation	Resource / Model	Details
READ	Ride/TripHistory	Aggregate rides: GROUP BY date, city; SUM fare; COUNT status
READ	Driver	JOIN rides; aggregate per driver for performance table
READ	Vendor+Vehicle	Fleet utilisation: active_vehicles/total_vehicles per vendor
READ	User	DAU (distinct user_ids today); MAU (distinct this month)

■ Acceptance Criteria / Definition of Done

- Revenue report matches Accountant payout totals (no discrepancy > ■1)

- Peak hours heatmap uses 15-minute buckets aggregated to hourly display
 - All reports exportable as CSV with proper column headers
 - Zone report overlays on map with colour intensity = demand level
 - Compliance report shows: docs expiring in 7/30 days, BGV pending count
-

Task 10.1

Broadcast Notifications — Segmented Push, SMS & In-App

Priority: Medium

Est: 2 days

Description

Manager/Support creates broadcast notifications: title, message, target audience (users/drivers/vendors/all), city filter, last-active-days filter, scheduled time. Channels: push (FCM), SMS, in-app. Delivery report shows target count and delivered count. Existing NotificationsPage.jsx must be fully built.

■ Screen Names & Navigation Path

- /notifications → NotificationsPage.jsx — Compose panel + sent history table
- Compose Form — Title, message (250 chars), audience selector, channel checkboxes, schedule datetime
- Audience Preview — 'Target Count: 12,430 users' shown before send
- Notification History Table — sent_at, title, channel, target_count, delivered_count, status
- Template Manager — Create/edit SMS and push templates with variables

Inputs	Outputs / Expected Results
• title (string), message (string), target: {audience, city, filters: {last_active_days}}	• notification_id, target_count, scheduled_at, status: 'scheduled' 'sent' 'failed'
• channels: ['push', 'in_app', 'sms'], scheduled_at (ISO datetime or null for immediate)	• Delivery report: delivered_count, failed_count, open_rate (push)

■ API Endpoints

Method	Endpoint	Auth Required	Description
POST	/api/v1/admin/notifications/broadcast	Manager/Support+	Create broadcast notification
GET	/api/v1/admin/notifications	Manager+	List sent/scheduled notifications
GET	/api/v1/admin/notifications/:id	Manager+	Notification detail + delivery stats
DELETE	/api/v1/admin/notifications/:id	Manager+	Cancel scheduled notification
POST	/api/v1/admin/notifications/templates	Manager+	Create SMS/push template
GET	/api/v1/admin/notifications/templates	Manager+	List templates
PATCH	/api/v1/admin/notifications/templates/:id	Manager+	Update template

■ CRUD Operations Required

Operation	Resource / Model	Details
CREATE	Notification	Insert broadcast record with target criteria
READ	Notification	List with status filter; paginate
DELETE	Notification	Cancel if status=scheduled and scheduled_at > now
CREATE	NotificationTemplate	Insert reusable template with variable placeholders
UPDATE	NotificationTemplate	Edit template content
DELETE	NotificationTemplate	Remove template

■ Acceptance Criteria / Definition of Done

- FCM batch send: use FCM v1 API; max 500 tokens per batch request
- SMS via Fast2SMS (India) or Twilio; rate limited to 100 SMS/minute
- Scheduled notifications processed by Celery beat at exact scheduled_at time

- Delivery stats updated via FCM delivery receipts and SMS callback
 - Vendor can only send notifications to their own drivers (own fleet segment)
-

Task 11.1

Vehicle Registry, Live Map & Maintenance Alerts

Priority: Medium

Est: 2 days

Description

Admin/Manager views all platform vehicles in a searchable registry and a live map (color-coded by status: on_trip=green, available=blue, offline=gray, suspended=red). Maintenance alerts for insurance/fitness cert expiring within 30 days. Existing FleetPage.jsx stub must show the registry table and integrate a map view.

■ Screen Names & Navigation Path

- /fleet → FleetPage.jsx — Toggle: 'Table View' | 'Map View'
- Table View — Filter by type, vendor, city, status; columns: vehicle_no, type, model, vendor, driver, status, insurance_expiry, fitness_expiry
- Map View — Leaflet/Mapbox map; colored markers by status; click marker for vehicle card
- Maintenance Alerts Banner — List of vehicles with expiry < 30 days
- Vehicle Detail Drawer — Full info, document links, trip history (last 10), deactivate button
- Deactivate Modal — Reason dropdown; temporary/permanent toggle

Inputs	Outputs / Expected Results
• Filters: ?vehicle_type=&vendor;_id=&city;=&status;=&expiry;_alert=true	• vehicles[]: vehicle_id, number, type, model, owner_type, owner_id, current_driver, status, lat, lng, insurance_expiry, fitness_expiry, expiry_alert
• Deactivate: vehicle_id, reason, is_permanent (bool)	• Maintenance alerts: [{vehicle_id, vehicle_number, alert_type, expiry_date, days_remaining}]

■ API Endpoints

Method	Endpoint	Auth Required	Description
GET	/api/v1/admin/vehicles	Manager+	Vehicle registry with filters
GET	/api/v1/admin/vehicles/:id	Manager+	Vehicle detail + trip history
POST	/api/v1/admin/vehicles/:id/deactivate	Manager+	Deactivate vehicle
POST	/api/v1/admin/vehicles/:id/activate	Manager+	Re-activate vehicle
GET	/api/v1/admin/vehicles/maintenance-alerts	Manager+	Vehicles with expiry < 30 days
GET	/api/v1/admin/vehicles/live-locations	Manager+	All active vehicles with lat/lng

■ CRUD Operations Required

Operation	Resource / Model	Details
READ	Vehicle	Paginated list; JOIN Vendor + Driver; filter multi-column
UPDATE	Vehicle	Deactivate/activate: update status, deactivation_reason, deactivated_at
READ	Ride	Last 10 trips per vehicle for trip history

■ Acceptance Criteria / Definition of Done

- Live map positions update every 30 seconds via polling (or WebSocket if available)
- Maintenance alert badge shows count in sidebar nav for /fleet
- Deactivated vehicle automatically unlinked from assigned driver
- Vehicles with expired insurance are auto-flagged and cannot accept new trips

Implement all VAPT-required security controls (OWASP Top-10) and the immutable audit log system. Super Admin and Auditor can view a searchable audit trail of every admin panel action. Existing AuditPage.jsx stub must show the full audit log with filters.

Task 12.1 Immutable Audit Log System**Priority: Critical****Est: 2 days****Description**

Every admin panel action is logged immutably: WHO (user, role, IP, device), WHAT (action, resource, ID), WHEN (UTC microsecond timestamp), RESULT (success/failure), BEFORE/AFTER (changed field snapshot). Logs are write-only (no UPDATE or DELETE on audit_log table). Existing audit.py middleware to be extended.

■ Screen Names & Navigation Path

- /audit → AuditPage.jsx — Searchable, filterable log table
- Filters: date range, admin user, role, action type, resource type, result (success/fail)
- Log Entry Detail Drawer — Full before/after JSON diff, IP, user-agent, device fingerprint
- Export button — Download filtered logs as CSV

Inputs	Outputs / Expected Results
• Every backend action auto-logs via middleware (no manual trigger)	• log_id, admin_user_id, role, ip_address, user_agent, action, resource_type, resource_id, before (JSON), after (JSON), result, timestamp
• Query filters: ?from=&to;=&user;_id=&action;=&resource;_t ype=&result;=&page;=&limit;=	•

■ API Endpoints

Method	Endpoint	Auth Required	Description
GET	/api/v1/admin/audit-logs	Super Admin / Auditor	List audit logs with filters
GET	/api/v1/admin/audit-logs/:id	Super Admin / Auditor	Single audit log entry detail
GET	/api/v1/admin/audit-logs/export	Super Admin	Download filtered logs as CSV

■ CRUD Operations Required

Operation	Resource / Model	Details
CREATE	AuditLog	INSERT only — no UPDATE or DELETE allowed on this table ever
READ	AuditLog	Paginated, filterable, sortable by timestamp desc

■ Acceptance Criteria / Definition of Done

- Audit log INSERT happens before the action response is returned (synchronous)
- No PII (phone, Aadhaar, PAN) in plain text in audit logs — masked
- Log export CSV matches filtered view exactly
- READ-ONLY auditor role can view and export audit logs but nothing else
- Database-level trigger prevents UPDATE/DELETE on audit_logs table

Task 12.2 VAPT Security Controls — API, Auth & Data**Priority: Critical****Est: 2 days**

Description	Implement all OWASP Top-10 controls: rate limiting on auth/API endpoints, CSRF protection, XSS prevention (CSP headers), SQL injection prevention (ORM only), IDOR prevention (ownership checks), AES-256 encryption for PII at rest, presigned S3 URLs, brute-force lockout, IP whitelisting config. Security Dashboard page for monitoring.
--------------------	---

■ **Screen Names & Navigation Path**

- (Super Admin only) Security Dashboard section in /settings or /audit
- Threat Monitor tab — Failed logins (last 24h), rate-limit hits, unusual IPs (geofencing alerts)
- IP Whitelist Management — Add/remove allowed IP ranges for admin panel access

Inputs	Outputs / Expected Results
• IP whitelist rules: CIDR notation entries	• 200 on successful requests; 429 Too Many Requests with Retry-After header on rate limit
• Security config: brute_force_attempts (default 5), lockout_minutes (default 30), rate_limit_auth (10/min), rate_limit_api (200/min/user)	• 403 on IP whitelist violation; 401 on auth failures

■ **API Endpoints**

Method	Endpoint	Auth Required	Description
GET	/api/v1/admin/security/dashboard	Super Admin	Threat monitoring overview
GET	/api/v1/admin/security/failed-logins	Super Admin	Failed login attempts
POST	/api/v1/admin/security/ip-whitelist	Super Admin	Add IP range to whitelist
DELETE	/api/v1/admin/security/ip-whitelist/:id	Super Admin	Remove IP from whitelist

■ **CRUD Operations Required**

Operation	Resource / Model	Details
CREATE	IPWhitelistRule	Insert allowed IP CIDR range
DELETE	IPWhitelistRule	Remove IP range
READ	LoginAttemptLog	Aggregate failed login attempts by IP / user for dashboard

■ **Acceptance Criteria / Definition of Done**

- All API responses include: X-Content-Type-Options, X-Frame-Options, Strict-Transport-Security, Content-Security-Policy headers
- Aadhaar, PAN, bank account numbers encrypted with AES-256 before DB storage
- KYC docs only accessible via 10-minute presigned S3 URLs — never public
- Rate limiter implemented via Flask-Limiter + Redis; config hot-reloadable
- VAPT quarterly test checklist document prepared and CERT-IN vendor shortlisted

14. GitHub Workflow — Merging All Modules into One Admin

This section defines exactly how the 5-member team collaborates on GitHub to build isolated feature modules and merge them into a single production-ready admin panel without conflicts. Follow this workflow strictly.

Branch Strategy

Branch	Purpose	Owner	Protected?
main	Production releases only; tagged versions	All (PR required)	YES
develop	Integration branch; all features merge here first	All (PR required)	YES
feature/m1-auth-rbac	Member 1: Auth, RBAC, Super Admin, Security	Arjun	No
feature/m2-driver-vendor-fleet	Member 2: Driver, Vendor, Fleet	Priya	No
feature/m3-fare-finance-analytics	Member 3: Fare, Finance, Analytics, Notifs	Kiran	No
feature/m4-frontend-auth-drivers	Member 4: Frontend Auth, Dashboard, Drivers, Vendors, Sidesa	Sneha	No
feature/m5-frontend-finance-support	Member 5: Frontend Finance, Support, Fleet, Analytics, Notifs, Audit	Vikram	No
hotfix/*	Emergency production fixes	Tech Lead	No

Step-by-Step Workflow

Step 1	Repository Setup (Arjun — Day 1)
	1. Create GitHub repo: github.com/novelski/railcab-admin 2. Push the existing code from railcab_admin.zip as the initial commit on main 3. Create develop branch from main: git checkout -b develop && git push origin develop 4. Set branch protection rules in GitHub Settings → Branches: • main: require 2 PR reviews; require CI pass; no direct push • develop: require 1 PR review; require CI pass 5. Add .github/CODEOWNERS file mapping directories to reviewers 6. All members clone: git clone https://github.com/novelski/railcab-admin.git
Step 2	Create Feature Branches (Each Member — Day 1)
	git checkout develop && git pull origin develop git checkout -b feature/m1-auth-rbac # Arjun git checkout -b feature/m2-driver-vendor-fleet # Priya git checkout -b feature/m3-fare-finance-analytics # Kiran git checkout -b feature/m4-frontend-auth-drivers # Sneha git checkout -b feature/m5-frontend-finance-support # Vikram
Step 3	Daily Development Protocol (All Members)
	Morning: git fetch origin && git rebase origin/develop (keep up with integration) Commit convention: type(scope): description Examples: feat(auth): add 2FA TOTP verification feat(drivers): implement KYC review workflow fix(fare): correct surge multiplier cap enforcement

test(support): add ticket SLA breach test
Push daily: git push origin feature/m1-auth-rbac

Step 4 File Ownership — Prevent Conflicts

Backend (Arjun M1 owns): backend/app/api/v1/routes/auth.py, admin_users.py, audit_logs.py, config.py
Backend (Priya M2 owns): backend/app/api/v1/routes/drivers.py, vendors.py, fleet routes
Backend (Kiran M3 owns): backend/app/api/v1/routes/fare.py, finance.py, analytics.py, notifications.py, support.py
Frontend (Sneha M4 owns): frontend/src/pages/auth/, dashboard/, drivers/, vendors/, rides/, admin-users/
Frontend (Vikram M5 owns): frontend/src/pages/finance/, support/, fare/, fleet/, analytics/, notifications/, audit/, settings/
SHARED (coordinate before changing): App.jsx, Sidebar.jsx, AuthContext.jsx, api.js, router.py
Add CODEOWNERS: # frontend/src/pages/drivers/ @sneha-iyer

Step 5 Pull Request (PR) Process

When a task is complete: git push origin feature/m2-driver-vendor-fleet
Open PR on GitHub: base=develop, compare=feature/m2-driver-vendor-fleet
PR title: [M2] feat: Driver onboarding + KYC approval workflow
PR description must include: What was done, Screenshots of UI, API endpoints added, DB migrations run
Assign 1 reviewer (Arjun reviews all backend PRs; Sneha reviews all frontend PRs)
CI/CD must pass: lint (ESLint/flake8), tests (pytest), build (npm run build)
After approval: Squash and merge into develop
Delete feature branch after merge

Step 6 Shared Backend Router — Merging Routes (Arjun)

backend/app/api/v1/router.py is the central router — Arjun owns it
When each member's backend PR is merged, Arjun adds their router include:
from app.api.v1.routes import drivers, vendors, fare, finance, support
router.include_router(drivers.router) # After M2 PR merged
router.include_router(fare.router) # After M3 PR merged
Each member documents their router prefix in their PR description

Step 7 Frontend Routing — Merging Pages (Sneha)

frontend/src/App.jsx is the central router — Sneha owns it
When Vikram's (M5) frontend PR is merged, Sneha merges App.jsx routes together:
Vikram's routes already exist in App.jsx stub; just need real page components
Sidebar.jsx navItems are already defined; both M4 and M5 work from same Sidebar
If App.jsx conflict occurs: git checkout develop -- frontend/src/App.jsx
Then manually add missing routes back, review diff with: git diff HEAD

Step 8 Database Migrations — Coordination

All DB models live in backend/app/models/; each member adds their model file
M1 owns: admin_user.py, audit_log.py, platform_config.py
M2 owns: driver.py (extend), vendor.py (extend), vehicle.py (new), driver_vehicle_link.py
M3 owns: fare_config.py, payout_batch.py, invoice.py, support_ticket.py, notification.py
Generate migration: alembic revision --autogenerate -m 'add vehicle table'
Run in order after each PR merge to develop:
1. M1 migrations (auth tables), 2. M2 migrations, 3. M3 migrations
NEVER run migrations directly on main — only on develop after PR merge
Migration files committed in the same PR as the model

Step 9 Integration Testing on Develop

After all 5 feature branches merged into develop:
1. Arjun runs all backend tests: cd backend && pytest tests/ -v
2. Sneha runs frontend build: cd frontend && npm run build

3. Full E2E test: login → dashboard → driver approval → fare config → payout
4. Test all RBAC permissions: login as manager, verify cannot access /settings
5. Fix integration issues on develop directly (small hotfixes only)

Step 10 Production Release (main)

Create PR: develop → main
All 5 members must approve this PR
CI runs full test suite + security scan (Snyk)
Tag the release: `git tag -a v1.0.0 -m 'RailCab Admin v1.0 — Phase 1'`
Deploy via existing `.github/workflows/deploy.yml` pipeline
Monitor error rates in Sentry for 30 minutes post-deploy

Common Merge Conflicts & Resolutions

File	Who Might Conflict	Resolution Strategy
App.jsx (frontend router)	Sneha + Vikram	Sneha owns; Vikram adds imports only in his PR; manual merge
Sidebar.jsx	Sneha + Vikram	Already complete in stub; no changes needed — do NOT edit
api.js (axios service)	Sneha + Vikram	Each adds only their API calls in their section; comment // M4 or // M5
router.py (backend)	All backend members	Arjun owns; others add include_router lines at bottom only
requirements.txt	All backend members	All add to bottom; Arjun resolves duplicates on merge
alembic/versions/*	M1, M2, M3	Run in dependency order; prefix migration file with m1_, m2_, m3_
package.json (frontend)	Sneha + Vikram	No new dependencies without team discussion; Sneha resolves

CI/CD Pipeline (from existing deploy.yml)

The existing .github/workflows/deploy.yml pipeline will run the following jobs on every PR to develop or main:

- lint-backend: flake8 backend/app/ — PEP8 compliance; pylint score >= 8.0
- lint-frontend: eslint frontend/src/ — Airbnb style guide
- test-backend: pytest backend/tests/ --cov=app --cov-fail-under=70
- build-frontend: npm run build — must succeed with 0 errors
- security-scan: snyk test --all-projects — fail on high severity CVEs
- docker-build: docker build backend/ && docker build frontend/
- deploy-staging: (on develop merge) deploy to staging environment
- deploy-production: (on main merge) deploy to production after manual approval gate

15. Appendix — Quick Reference

Tech Stack Reference

Layer	Technology	Version / Notes
Admin Frontend	React + Vite + TailwindCSS	React 18, Vite 5 — existing in /frontend
Admin Backend	FastAPI + SQLAlchemy + PostgreSQL	Python 3.11, async, existing in /backend
Cache / Sessions	Redis	Sessions, rate limits, surge cache, OTP storage
File Storage	AWS S3 / Cloudflare R2	KYC docs private bucket; presigned 10-min URLs
Payments	Razorpay API	Collections, Payout API, Webhooks (HMAC verified)
Push Notifications	Firebase Cloud Messaging (FCM)	v1 API; batch 500 tokens max
SMS	Fast2SMS / Twilio	OTP, ride confirmations, KYC updates
Background Jobs	Celery + Redis broker	Surge computation (5-min), invoice gen, scheduled notifs
2FA	pyotp (TOTP)	RFC 6238; Google Authenticator compatible
Encryption	AES-256-GCM	PII at rest: Aadhaar, PAN, bank account
Maps	Leaflet.js / Mapbox GL	Live fleet map; geo-zone polygon drawing
Charts	Recharts / Chart.js	Already in frontend dependencies
CI/CD	GitHub Actions	Existing .github/workflows/deploy.yml

Existing Code — What's Already Built (from railcab_admin.zip)

File / Directory	Status	Notes
frontend/src/App.jsx	■ Stub	All routes defined; pages imported but mostly empty
frontend/src/components/layout/Sidebar.jsx	■ Complete	All nav items; permission-gated display
frontend/src/contexts/AuthContext.jsx	■ Partial	JWT storage done; canAccess() needs permission map
frontend/src/pages/auth/LoginPage.jsx	■ Partial	Form UI done; API call needs wiring
frontend/src/pages/dashboard/DashboardPage.jsx	■■ Stub	Layout done; needs real API data
frontend/src/pages/admin-users/AdminUsersPage.jsx	■■ Partial	Table UI done; CRUD modals incomplete
frontend/src/pages/drivers/DriversPage.jsx	■■ Stub	Basic table only; KYC review drawer missing
frontend/src/pages/vendors/VendorsPage.jsx	■■ Stub	Table only; approval workflow missing
frontend/src/pages/rides/RidesPage.jsx	■■ Stub	Table only; detail drawer missing
frontend/src/pages/finance/FinancePage.jsx	■■ Stub	Tab structure only; no real data
frontend/src/pages/support/SupportPage.jsx	■■ Stub	Basic layout only
frontend/src/pages/fare/FareEnginePage.jsx	■■ Partial	Form layout done; API not wired
frontend/src/pages/fleet/FleetPage.jsx	■■ Stub	Table only; map view missing
frontend/src/pages/analytics/AnalyticsPage.jsx	■■ Stub	Placeholder charts only
frontend/src/pages/notifications/NotificationsPage.jsx	■■ Stub	Form structure only

File / Directory	Status	Notes
frontend/src/pages/audit/AuditPage.jsx	■■ Stub	Basic table; filters missing
frontend/src/services/api.js	■ Partial	Axios instance with JWT interceptor; needs all API calls
backend/app/api/v1/routes/auth.py	■ Partial	Login + 2FA flow; needs session invalidation
backend/app/api/v1/routes/admin_users.py	■ Partial	CRUD done; invite email needs wiring
backend/app/api/v1/routes/drivers.py	■ Partial	Approval endpoints; missing BGV/deposit
backend/app/api/v1/routes/vendors.py	■ Partial	Registration done; fleet management missing
backend/app/api/v1/routes/fare.py	■ Partial	Config GET/PATCH; surge algorithm missing
backend/app/api/v1/routes/finance.py	■■ Stub	Empty stubs only
backend/app/api/v1/routes/support.py	■■ Stub	Basic ticket create/list
backend/app/api/v1/routes/analytics.py	■■ Stub	Empty
backend/app/models/driver.py	■ Complete	Full Driver model with all fields and status enum
backend/app/models/vendor.py	■ Partial	Vendor model; Vehicle model missing
backend/app/models/admin_user.py	■ Complete	AdminUser with role, permissions, 2FA fields
backend/app/models/audit_log.py	■ Complete	AuditLog model with all required fields
backend/app/middleware/audit.py	■■ Partial	log_action helper; needs full before/after capture
backend/app/middleware/security.py	■ Partial	Rate limiting; CORS; needs CSP headers
backend/app/core/security.py	■ Complete	JWT, password hashing, TOTP generation
backend/app/core/deps.py	■ Complete	Role dependencies: ManagerRequired, AccountantRequired etc.
.github/workflows/deploy.yml	■ Complete	Full CI/CD pipeline; deploy steps ready

Estimated Delivery Timeline

Week	Arjun (M1)	Priya (M2)	Kiran (M3)	Sneha (M4)	Vikram (M5)
Week 1 (Days 1–5)	Auth system, 2FA, JWT	Driver registration, OTP, doc upload	Fare config & surge algorithm	Login page, dashboard, RBAC UI	Finance page setup
Week 2 (Days 6–10)	RBAC, roles, admin user CRUD	KYC review, approval workflow	Finance: earnings, payouts, GST	Driver approval drawer, vendor page	Support tickets, audit page
Week 3 (Days 11–15)	Platform config, VAPT controls	Vendor fleet, vehicle mgmt	Support, analytics, notifications	Ride management, all-pages polish	Fleet map, analytics charts, notifications
Week 4 (Days 16–20)	Audit system, security dashboard	Fleet page, live map integration	Invoice PDF, export functions	E2E integration, bug fixes	Full polish, E2E integration
Week 5 (Days 21–25)	Code review, CI/CD final	Code review, testing	Code review, testing	Code review, cross-browser test	Code review, mobile responsive

CONFIDENTIAL: This document is the complete task specification for the RailCab Admin Panel Phase 1 development. All 10 modules, all screens, all APIs, and all CRUD operations defined herein must be implemented before production release. No feature deferral is permitted for Phase 1. Questions should be directed to the Tech Lead (Arjun Reddy) or the Project Manager at Novelski Technologies.